

Spark Structured Streaming

Emanuele Della Valle

Prof @ PoliMI

CRO & founder @ Motus ml

founder @ Quantia Consulting

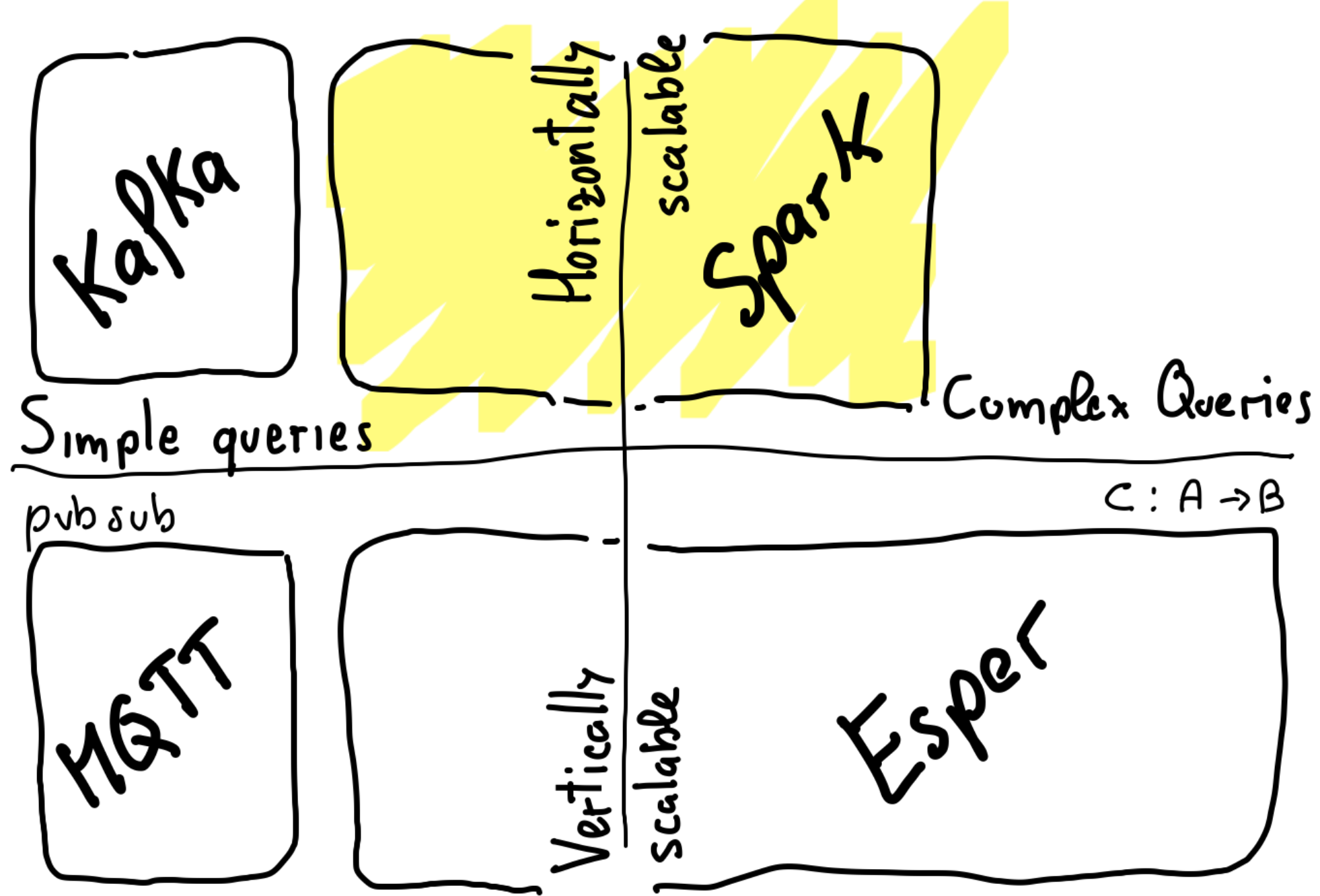


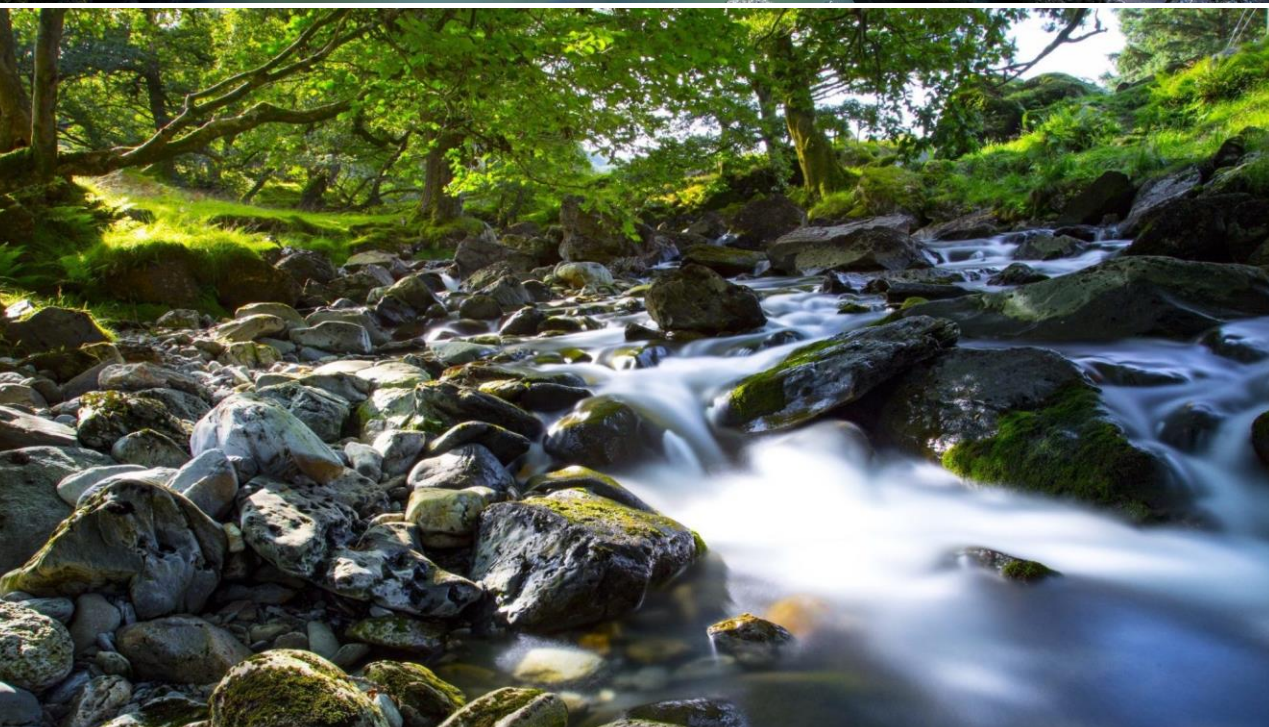
POLITECNICO
MILANO 1863



POLITECNICO
MILANO 1863

Positioning this lecture



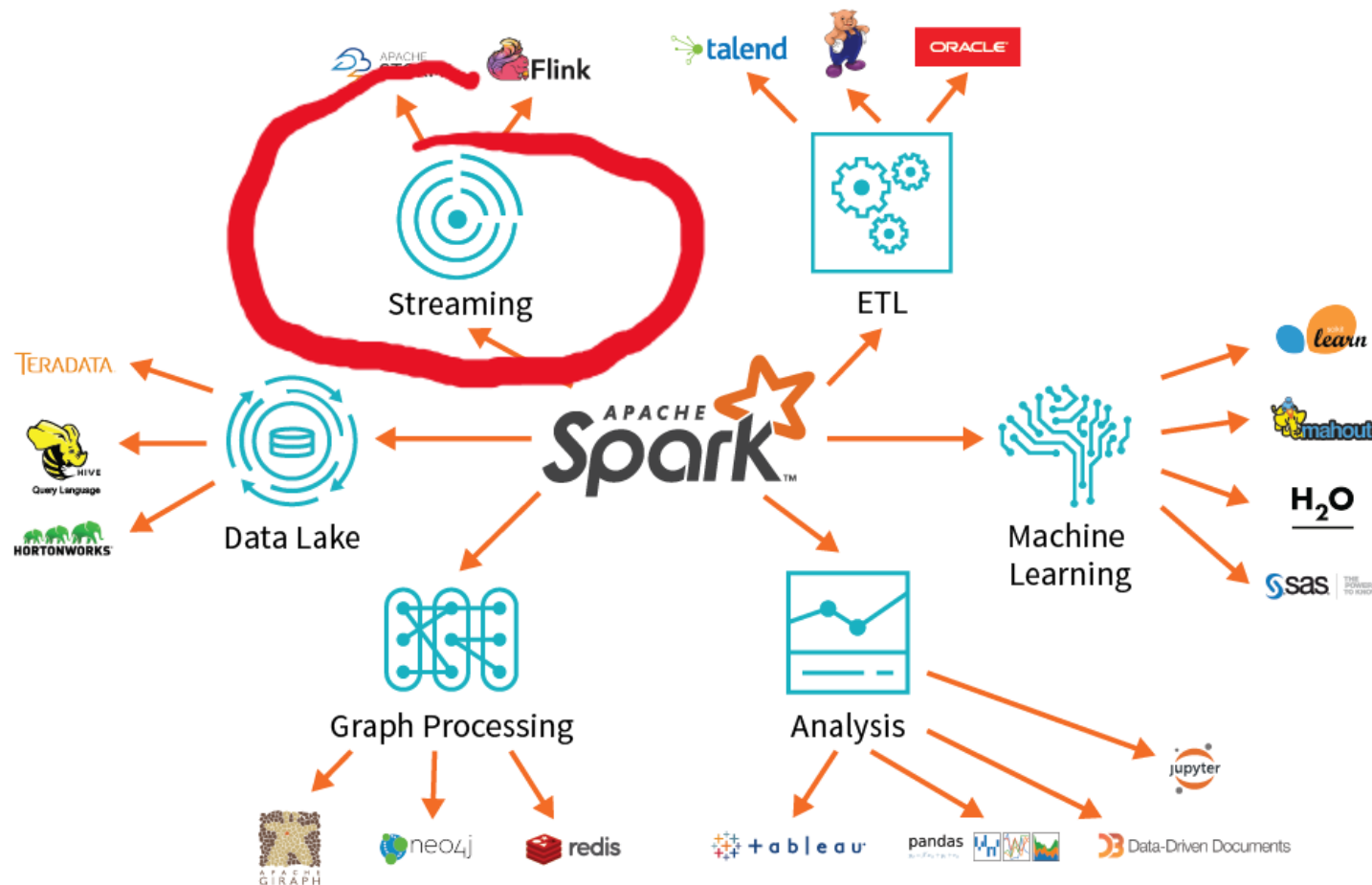




POLITECNICO
MILANO 1863



Unified analytics
engine for big data
processing, with
built-in modules for
streaming, SQL,
machine learning
and graph
processing



[src: <https://posts.specterops.io/threat-hunting-with-jupyter-notebooks-part-3-querying-elasticsearch-via-apache-spark-670054cd9d47>]

Where does come from?

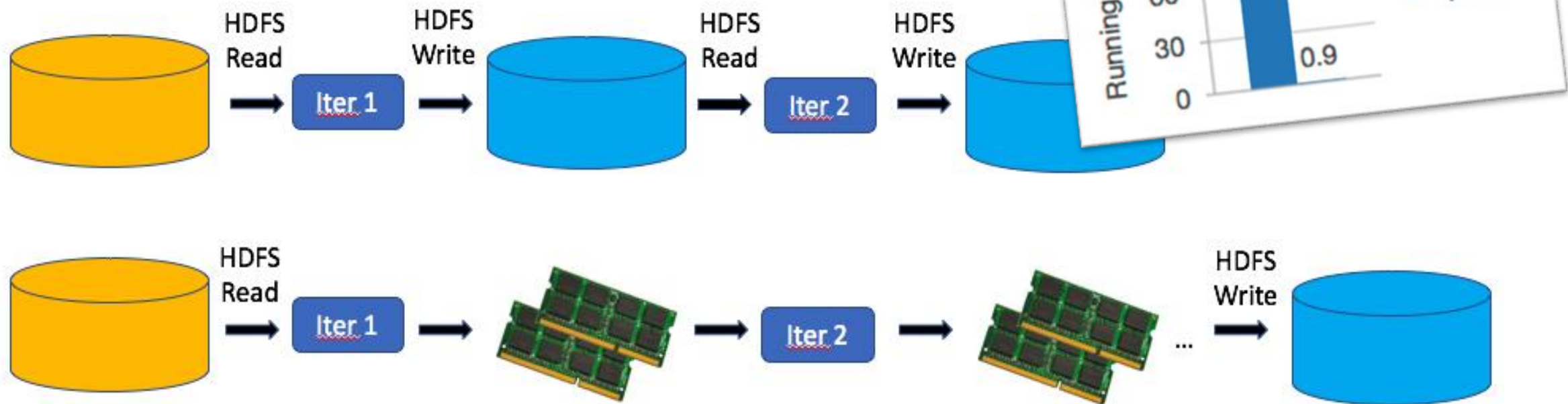
Apache Spark is the Taylor Swift of big data software. The open source technology has been around and popular for a few years. But 2015 was the year Spark went from an ascendant technology to a bona fide superstar.

-- Derrick Harris, 2015 on Fortune

- in 2009, Matei Zaharia started Spark as a research project at the University of California AMPLab
- In 2013, the project was donated to the Apache Software Foundation
- In February 2014, Spark became a Top-Level Apache Project
- In November 2014, Spark founder Matei Zaharia's company Databricks set a new world record in large scale sorting using Spark

The key ideas of

1st use RAM instead of disk



The key ideas of

2nd Ease of use

- E.g., WordCount in java vs. RDD (low level APIs)

```
text_file.flatMap(lambda line: line.split())  
  
    .map(lambda word: (word, 1))  
  
    .reduceByKey(lambda a, b: a+b)
```



- Even easier with DataFrame APIs

```
df = spark.read.json("logs.json")  
df.where("age > 21")  
    .select("name.first").show()
```

```
package org.myorg;  
  
import java.io.IOException;  
import java.util.*;  
  
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.conf.*;  
import org.apache.hadoop.io.*;  
import org.apache.hadoop.mapreduce.*;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;  
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;  
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;  
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;  
  
public class WordCount {  
  
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {  
        private final static IntWritable one = new IntWritable(1);  
        private Text word = new Text();  
  
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {  
            String line = value.toString();  
            StringTokenizer tokenizer = new StringTokenizer(line);  
            while (tokenizer.hasMoreTokens()) {  
                word.set(tokenizer.nextToken());  
                context.write(word, one);  
            }  
        }  
    }  
  
    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {  
  
        public void reduce(Text key, Iterable<IntWritable> values, Context context)  
            throws IOException, InterruptedException {  
            int sum = 0;  
            for (IntWritable val : values) {  
                sum += val.get();  
            }  
            context.write(key, new IntWritable(sum));  
        }  
    }  
  
    public static void main(String[] args) throws Exception {  
        Configuration conf = new Configuration();  
  
        Job job = new Job(conf, "wordcount");  
  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
  
        job.setMapperClass(Map.class);  
        job.setReducerClass(Reduce.class);  
  
        job.setInputFormatClass(TextInputFormat.class);  
        job.setOutputFormatClass(TextOutputFormat.class);  
  
        FileInputFormat.addInputPath(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
        job.waitForCompletion(true);  
    }  
}
```

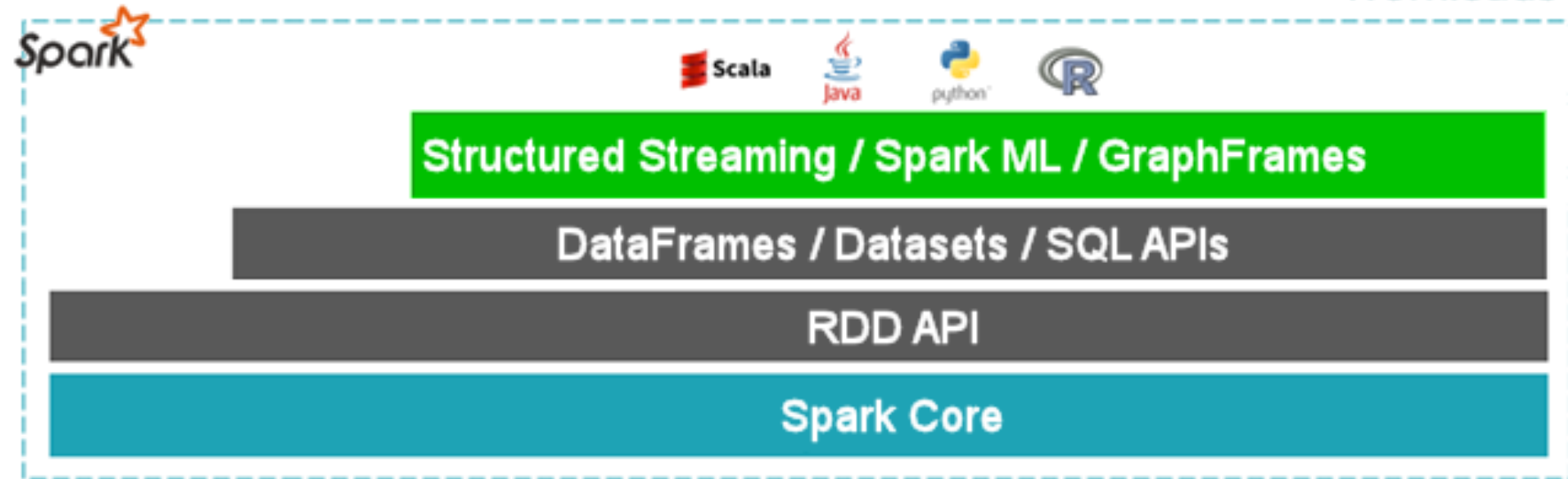
java

The key ideas of

3rd Generality

Environments

Workloads



RDD

- Definition

- **Resilient**: Fault-tolerant
- **Distributed**: Computed across multiple nodes
- **Dataset**: Collection of partitioned data

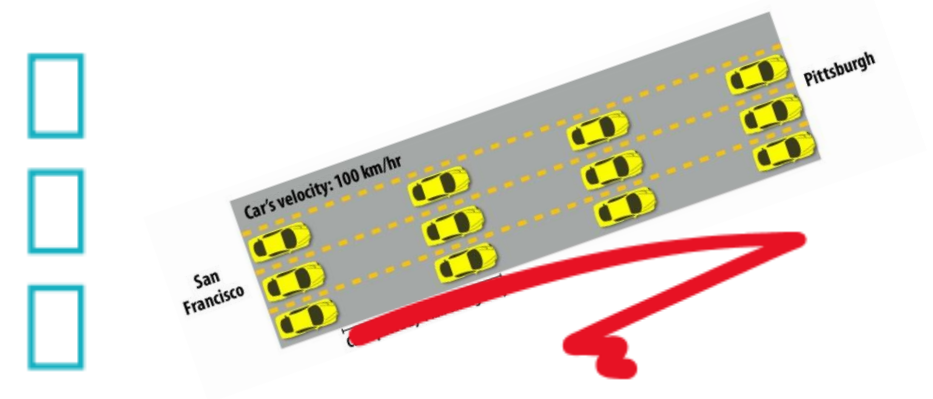
- Characteristics

- Immutable once constructed
- Track lineage information

- Logical view: single dataset



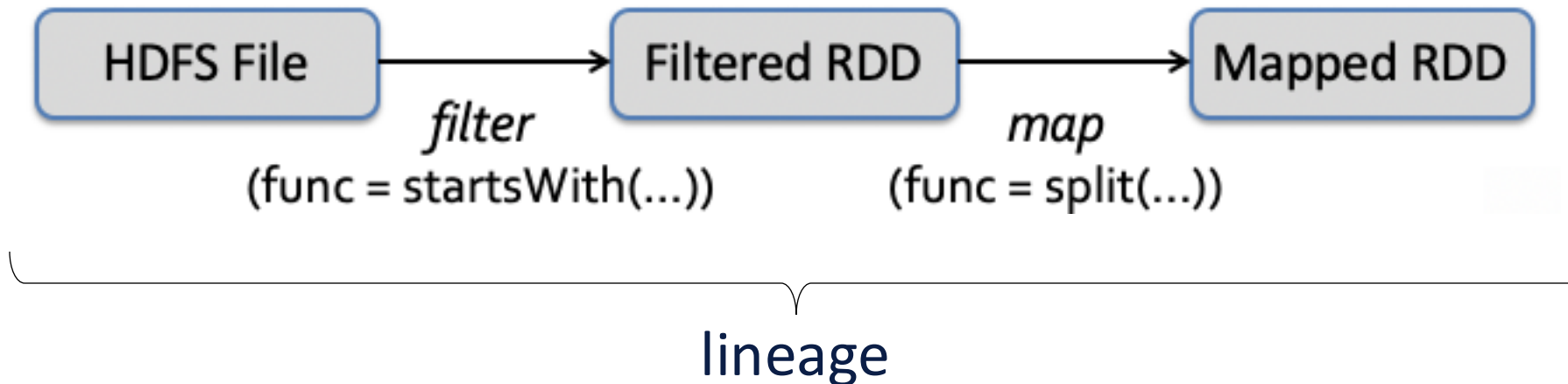
- Physical view: 1 or more partitions



Transformations - logical view

- RDD are immutable, transformations create new RDD

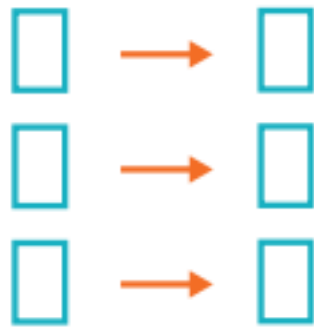
```
msgs = (textFile.filter(lambda s: s.startsWith("ERROR"))  
        .map(lambda s: s.split("\t")[2]))
```



Transformations – physical view

Narrow Transformations

The data required to compute the records in a single partition reside in at most one partition of the parent RDD



- `map()`
- `filter()`
- ...

Wide Transformations (shuffles)

The data required to compute the records in a single partition may reside in many partitions of the parent RDD



- `groupBy(...).sum()`
- `distinct()`
- ...

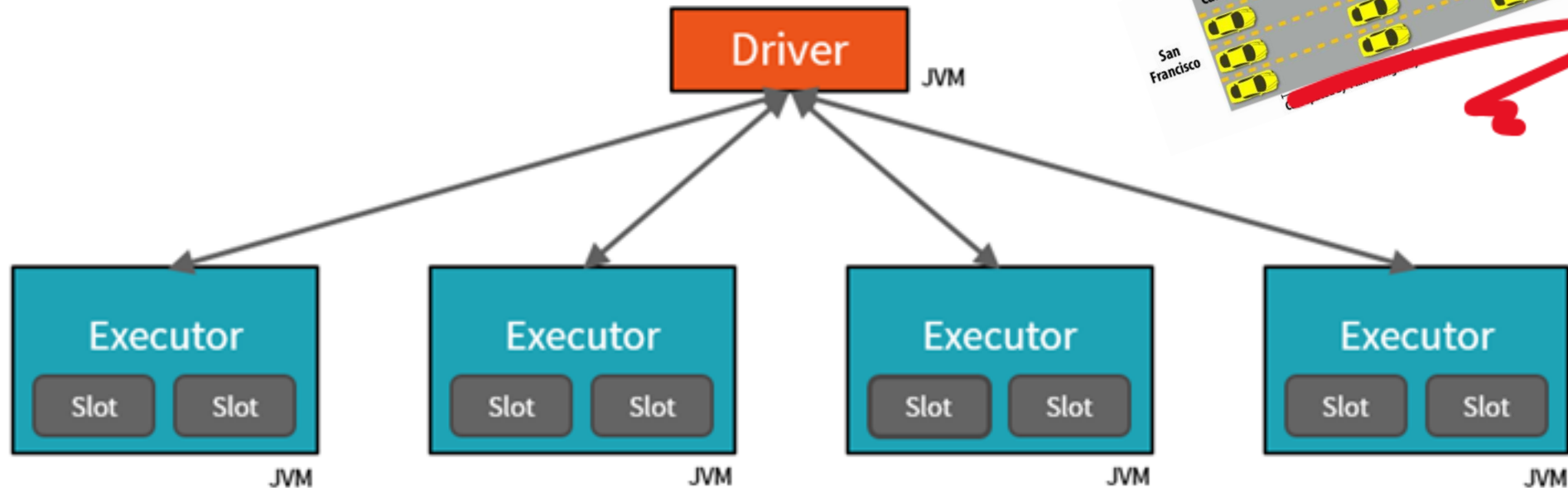


Actions – logical view

- Actions either return a result or write to disc
- For example:
 - The number of records in the case of `count()`
 - An array of objects in the case of `collect()` or `take(n)`

Spark at work - components

- A Spark cluster includes
 - One **Driver** running the user application
 - Multiple **Executors** (typically, one instance per node) with multiple **Slots** (typically, one per core/CPU)



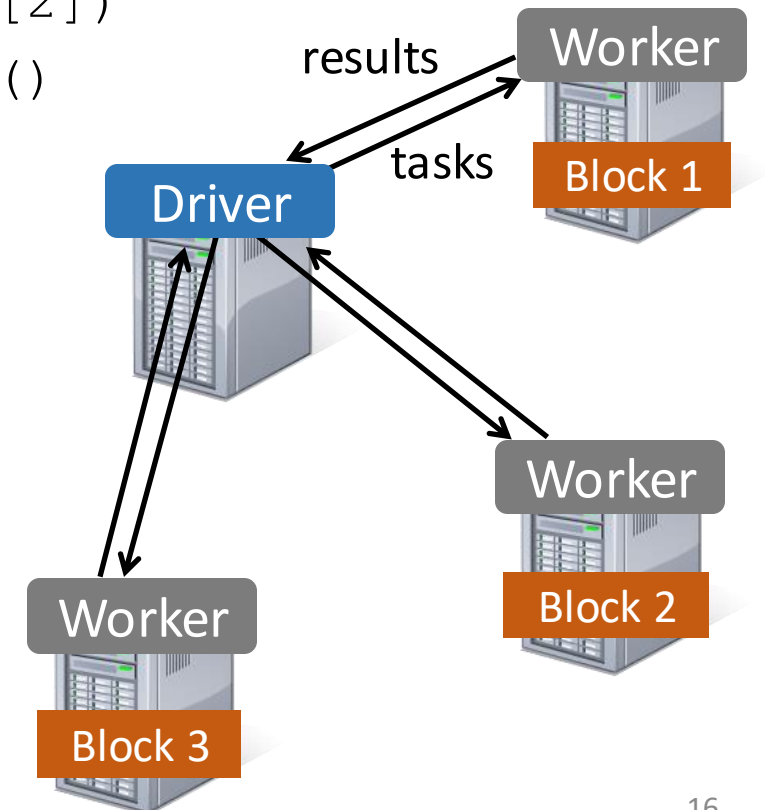
Actions – system view

Base RDD

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda s: s.startswith("ERROR"))  
messages = errors.map(lambda s: s.split("\t")[2])  
message.filter(lambda s: "mysql" in s).count()
```

Transformed RDD

Action

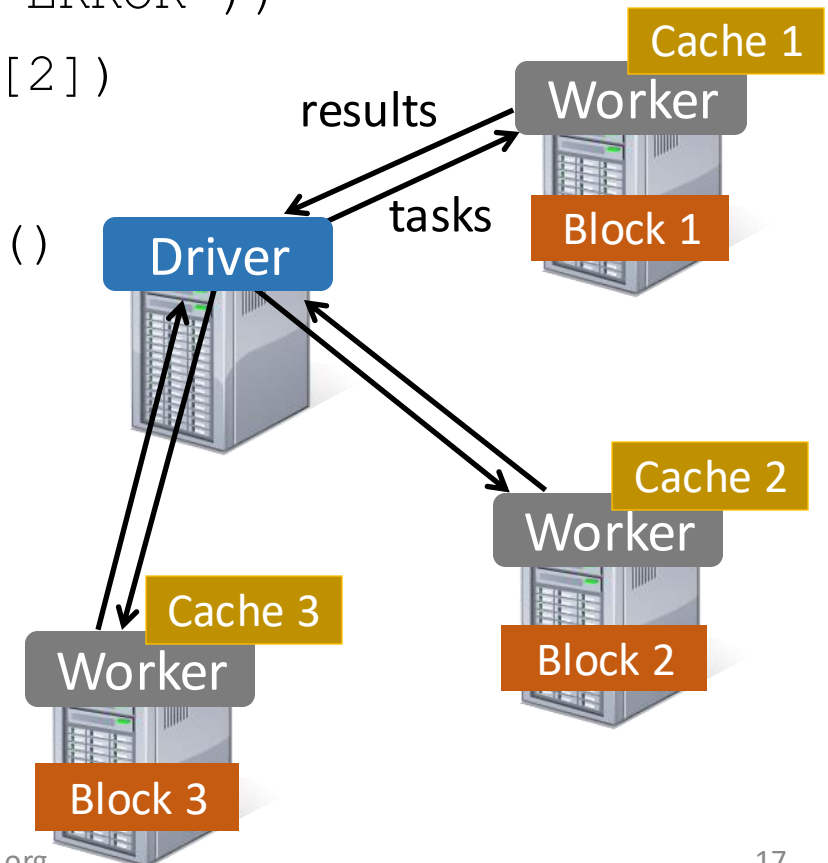


Caching (where the magic is!)

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
. . .
```

IN PRACTICE
Full-text search of Wikipedia

- 60GB on 20 EC2 machine
- 0.5 sec vs. 20s for on-disk





Laziness By Design

- Fundamental to Apache Spark are the notions that
 - Transformations are **LAZY**
 - Actions are **EAGER**
- NOTE: We saw this play out in the previous slide when we run multiple transformations back-to-back, and no job was triggered until the first action was requested

DataFrame

- Definition
 - Immutable Data with named columns
 - Built on RDDs
- Characteristics
 - User-friendly API
 - Uniform APIs across languages (Scala, Java, Python, R, and SQL)
 - Improved performance via optimizations (Tungsten and Catalyst)

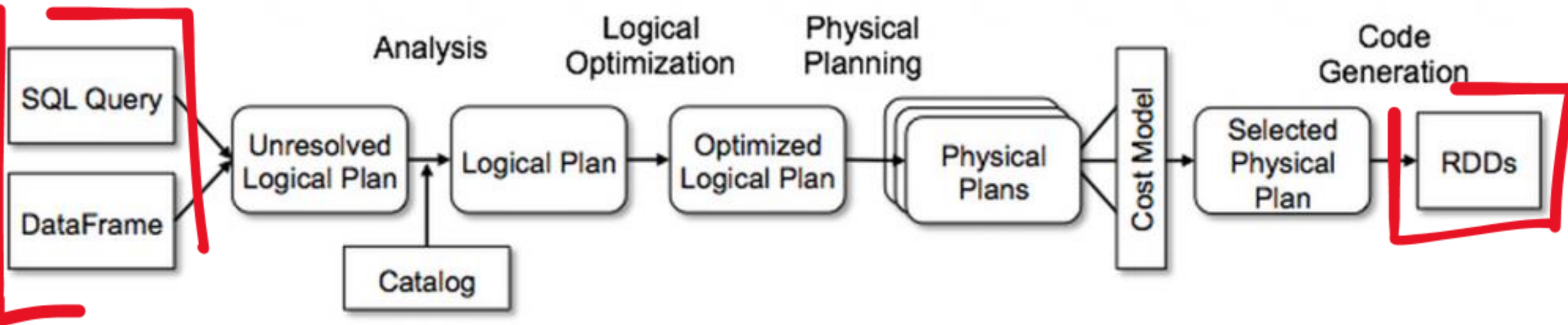
DataFrame code snippet

- An Apache Spark code snippet using SQL and DataFrames to query and join different data sources

```
# Read a CSV
userDF = spark.read.csv(" ... /userData.csv")
# ... or Use DataFrame APIs and register a temp view
middleageSmokers = userDF.filter(col("smoker")== "Y").filter(col("age")>40)
middleageSmokers.createOrReplaceTempView("middleageSmokers")
# ... read a CSV and register a temp view
spark.read.json(" ... /part-00000.json.gz").createOrReplaceTempView("iot_stream")
# ... execute SQL query or ...
spark.sql("SELECT avg(calories_burnt) FROM iot_stream JOIN middleageSmokers ON ...")
```

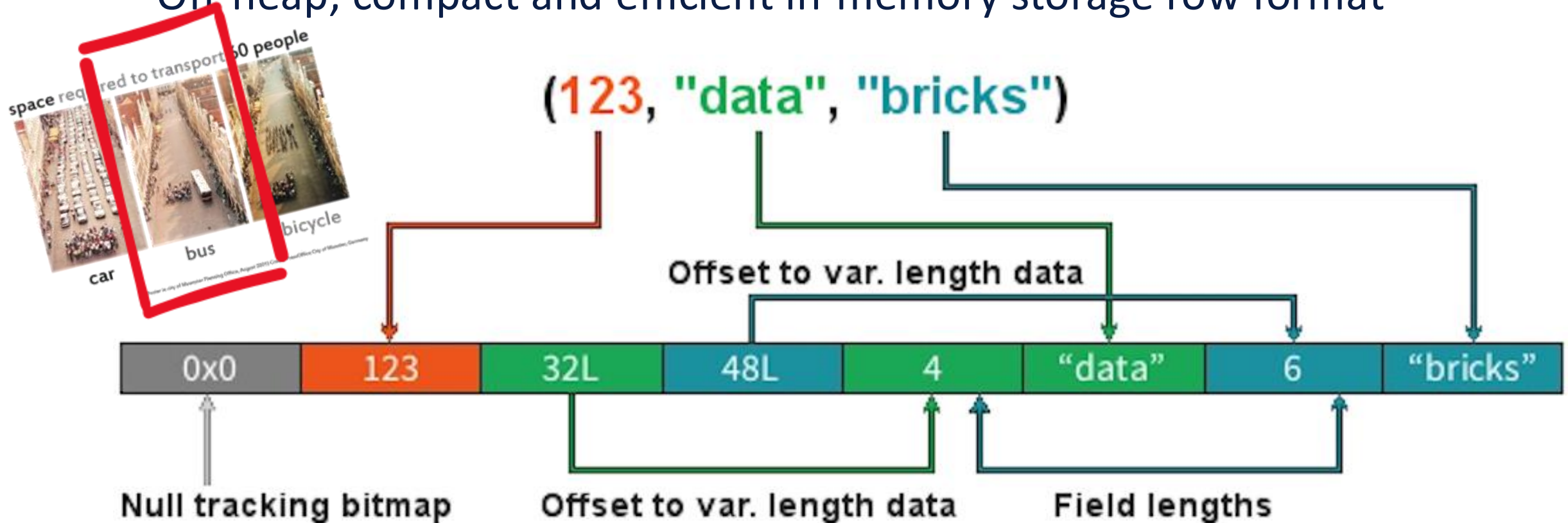
Under the Hood: Catalyst

- Generates optimal RDD code from the user request



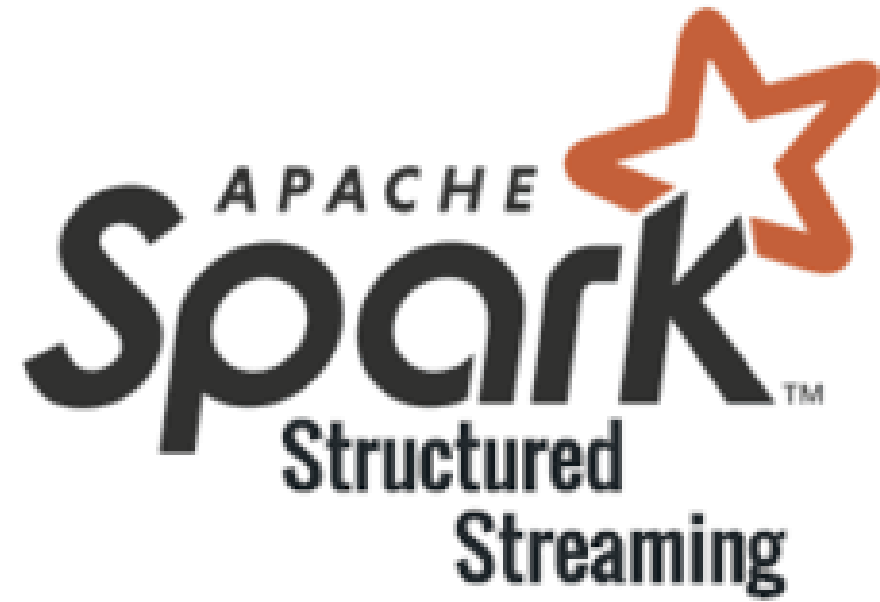
Under the Hood: Tungsten Binary Format

- Off-heap, compact and efficient in-memory storage row format





POLITECNICO
MILANO 1863





Overview

Spark Structured Streaming
provides fast, scalable, fault-tolerant,
end-to-end **exactly-once** stream processing
without the user **having to reason** about streaming.

Positioning Spark Structured Streaming

- default: micro-batch processing engine
 - end-to-end latencies as low as 100 milliseconds
 - exactly-once fault-tolerance guarantees
- since Spark 2.3, also Continuous Processing
 - end-to-end latencies as low as 1 millisecond
 - at-least-once guarantees
- choose the mode based on requirements

Programming Model

- key idea: treat a live data **stream as an unbounded table** that is being continuously appended
- Result:
 - user can **logically** express streaming computation **as** a DAG of standard batch-like queries **on static tables**
 - Spark **physically** runs the DAG **incrementally**

Modern DSMS

Logic View
Unbounded Table

K	V
♥	2
♥	7
♦	4
♥	3

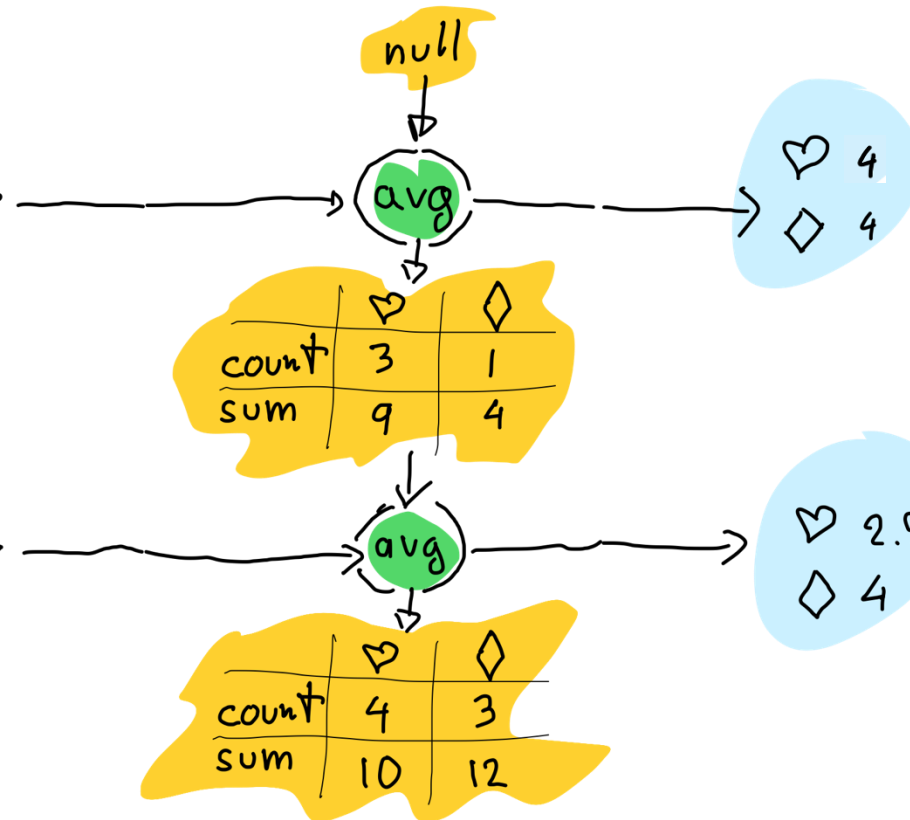
+

♦	2
♦	6
♥	2

+

⋮

Physical View
Incremental Evaluation



state

operator

results



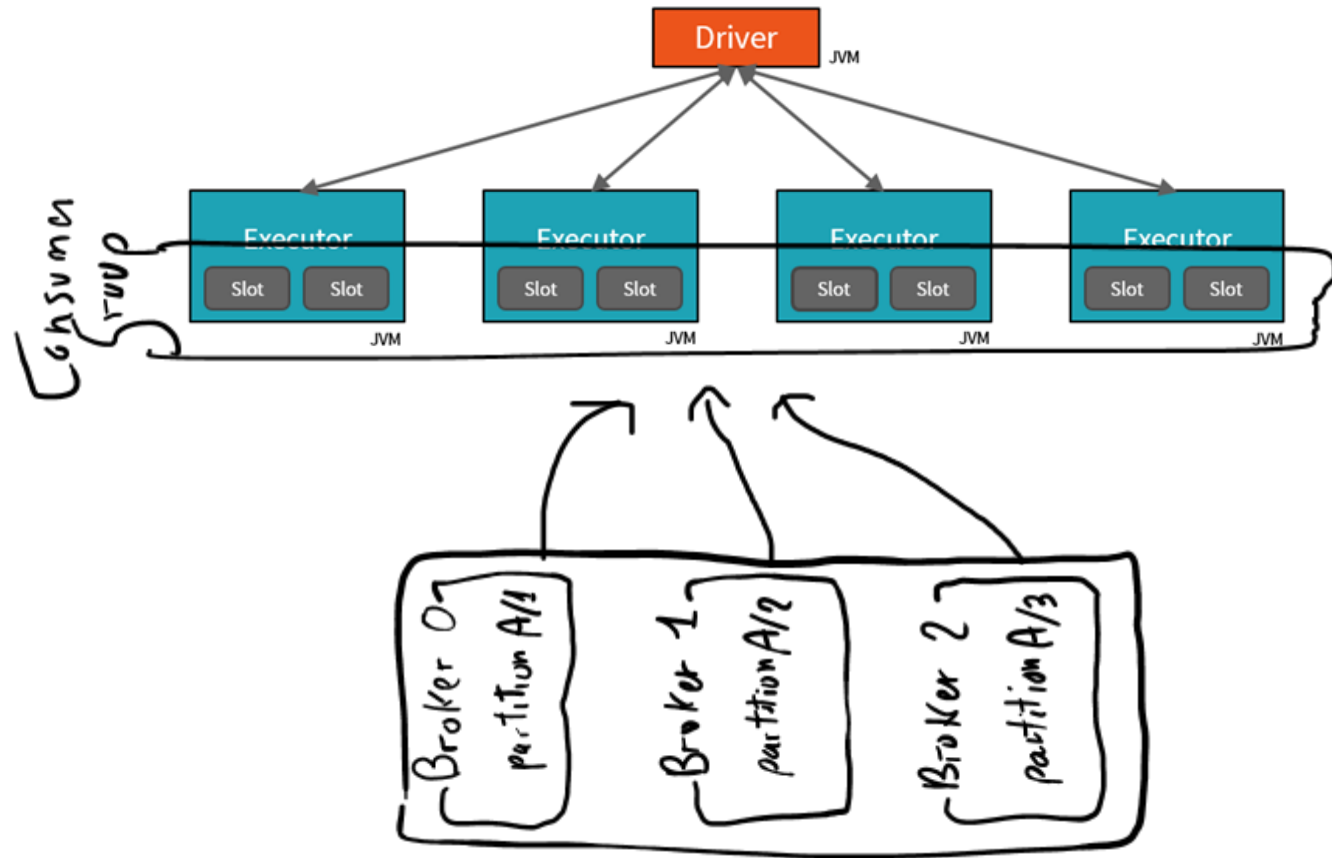
Programming Model (cont.)

- Spark Structured Streaming **does not materialize the entire table**
 - reads the latest available data from the stream
 - processes it incrementally to update the result
 - discards the source data
- It **only** keeps around the **minimal intermediate state data**
 - e.g. intermediate counts

Creating streaming DataFrames

- Streaming DataFrames can be created through the DataStreamReader interface returned by SparkSession.readStream().
- Input Sources:
 - File source, i.e., a folder in which new files get dropped
 - Supported file formats: text, CSV, JSON, ORC, Parquet
 - **Kafka** source
 - Reads data from Kafka
 - Socket source (for testing only)
 - Reads UTF8 text data from a socket connection.
 - It does not provide end-to-end fault-tolerance guarantees.

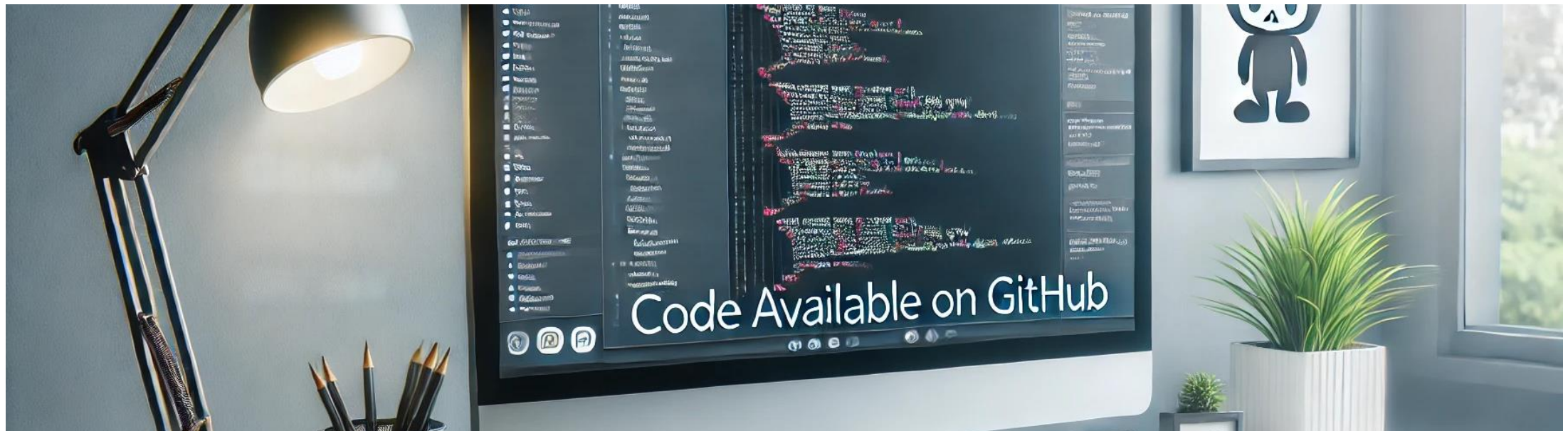
Connecting Spark to kafka



Code available on github!

- All the code described in this lecture is available at

https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/Spark/sss_firealarm



- **NOTE:** It requires HW (8 cores and 16-32GB RAM), docker, docker-compose and a good internet connection. **Don't try this for the first time in class!**

Creating streaming DataFrames (cont.)

- Streaming DataFrames can be created through the DataStreamReader interface returned by SparkSession.**readStream()**.

```
raw_sdf = (spark
  .readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", servers)
  .option("startingOffsets", "earliest")
  .option("subscribe", temperature_topic)
  .load())
```

Creating streaming DataFrames (cont.)

The schema of raw_sdf is the fixed on of every kafka topic

root

here are your data

```
|-- key: binary (nullable = true)
|-- value: binary (nullable = true)
|-- topic: string (nullable = true)
|-- partition: integer (nullable = true)
|-- offset: long (nullable = true)
|-- timestamp: timestamp (nullable = true)
|-- timestampType: integer (nullable = true)
```

Creating streaming DataFrames (cont.)

```
raw_sdf = (spark
  .readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", servers)
  .option("startingOffsets", "earliest")
  .option("subscribe", temperature_topic)
  .load())
```

```
casted_sdf = raw_sdf.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")
```

```
+---+-----+
```

```
|key|value|
```

```
+---+-----+
```

```
|S1 |{"sensor": "S1", "smoke": false, "ts": 1728893631}|
```

```
+---+-----+
```

*transform it
into a string*



← this is a json string

Creating streaming DataFrames (cont.)

```
temp_schema = StructType([  
    StructField("sensor", StringType(), True),  
    StructField("temperature", DoubleType(), True),  
    StructField("ts", TimestampType(), True)])
```

```
parsed_sdf = raw_sdf.select(from_json(col("value").cast("string"), temp_schema))
```

```
temperature_sdf = parsed_sdf.select("value.*")
```

provide a schema
to extract your data



if you want a table
flatten the tree-like
structure of a json



Basic Operations

Selection, and Projection

The dataframe syntax (works always)

```
filtered_sds = temperature_sdf.select("sensor").where("temperature > 50")
```

The SQL syntax on a view (works only for simple queries)

```
temperature_sdf.createTempView("TemperatureSensorEvent")
```

```
filtered_sds = (spark.sql("select * from TemperatureSensorEvent where temperature > 50"))
```

NOTE: The two syntaxes are equivalent, spark compiles down both of them



Basic Operations

Aggregation

The dataframe syntax (works always)

```
avg_temp = temperature_sdf.groupBy("sensor").avg()
```

The SQL syntax on a view (works only for simple queries)

```
query_string = """  
SELECT SENSOR, AVG(temperature)  
FROM TemperatureSensorEvent  
GROUP BY SENSOR  
"""
```

```
avg_temp = (spark.sql(query_string))
```

NOTE: The two syntaxes are equivalent, spark compiles down both of them

Spark is lazy ... Write results!

- Even streaming queries do not start running if you do not connect a sink and start writing on it
- To do that , use the DataStreamWriter returned through Dataset.**writeStream()**

Spark is lazy ... Write results! (cont.)

- specify:
 - **output sink**: where to write
 - **output mode**: when and what to write
 - query name: Optionally, a unique name of the query
 - trigger interval: Optionally, the trigger interval
 - checkpoint location: for the end-to-end fault-tolerance
- **Output Sinks**:
 - File sink - a directory
 - **Kafka sink** - one or more topics in Kafka
 - **Memory sink** (for debugging)
 - Console sink (for debugging)

Spark is lazy ... Write results! (cont.)

- **Append mode (default)**
 - only the **new rows** added to the result since the last trigger
 - only supported for queries whose output rows will **never change**
 - each row is output only once (assuming fault-tolerant sink)
 - e.g., queries with only select, where, map, flatMap, filter, join, etc.
- **Complete mode (dangerous!)**
 - the **whole result** will be outputted after every trigger
 - supported **only** for **aggregation queries**
- **Update mode**
 - Only the **rows** in the result that were **updated** since the last trigger
 - supported for **aggregation queries** and a few others, not for joins

Window Operations on Event Time

Spark Structured Streaming

- supports only logical windows on event time
- allows for declaring tumbling, hopping , and session windows
- treats windows as particular grouping criteria

temperature_sdf

```
.groupBy(window("TS", "1 minutes", "30 seconds"),"SENSOR")
```



Stream to Stream Join Operations

Inner	Supported , <i>optionally specify watermark on both sides + time constraints for state cleanup</i>
Left Outer	Conditionally supported, must specify watermark on right + time constraints for correct results, optionally specify watermark on left for all state cleanup
Right Outer	Conditionally supported, must specify watermark on left + time constraints for correct results, optionally specify watermark on right for all state cleanup
Full Outer	Conditionally supported, must specify watermark on one side + time constraints for correct results, optionally specify watermark on the other side for all state cleanup
Left Semi	Conditionally supported, must specify watermark on right + time constraints for correct results, optionally specify watermark on left for all state cleanup

«Followed by» operator as a stream to stream join

- Spark Structured Streaming does not support the EPL's operator -> (that reads as “followed by”).
- We need to use a stream-to-stream join with temporal constraints on the events' timestamps.

```
join_sdf = (smoke_events.join(  
  high_temperature_events, expr("""  
    (sensorTemp == sensorSmoke) AND  
    (tsTemp > tsSmoke ) AND  
    (tsTemp < tsSmoke + interval 2 minute )  
    """)  
  )))
```

← This is EPL's → operator

← This is EPL's where timer: within clause

It does not tame the torrent effect, but this is expected!

- The stream to stream join

```
join_sdf = (A.join(B, expr("""  
    (idA == idB) AND  
    (tsB > tsA ) AND  
    (tsB < tsA + interval 2 minute )  
    """))))
```

- Is equivalent to the EPL pattern

```
every x=A ->  
    every B(id=x.id)  
where timer:within(2 minutes)
```



Stream to Table Join Operations

- The result of the streaming join is generated incrementally

```
staticDf = spark.read. ...  
streamingDf = spark.readStream. ...  
streamingDf.join(staticDf, "type") # inner join  
streamingDf.join(staticDf, "type", "left_outer") # left outer join
```

- INNER and LEFT-OUTER stream-table joins are
 - **supported** and
 - **not stateful** :-D
- OUTER and RIGHT-OUTER stream-table joins are **not supported**

Late arrivals

- **late arrivals** refer to events that **arrive after the expected time** window in which they were supposed to be processed.
- This **happens because** in real-time streaming, data may experience delays due to
 - temporary disconnection
 - **network** latency
 - processing issues
 - ...



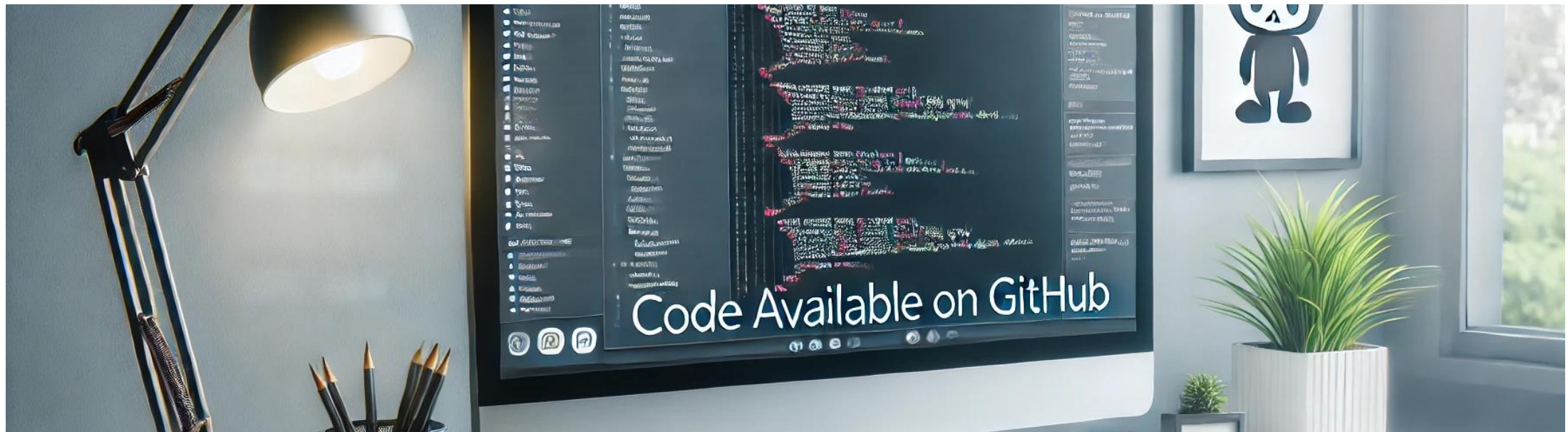
Event time, late arrivals , and windowed aggregation

- The **event-time** captures the time when the **data** was generated
- **rows may arrive late**, i.e., **out-of-order**, to the processing operator
- **watermarking**
 - tells, with **late-threshold T**, the system how long to wait for late arrivals
 - the system tracks the **processing time P**, i.e., the maximum seen event time
 - if the **difference** between the processing time and the event time of a later arrival is **smaller than T**, the event is **considered**, **otherwise** is **discarded**

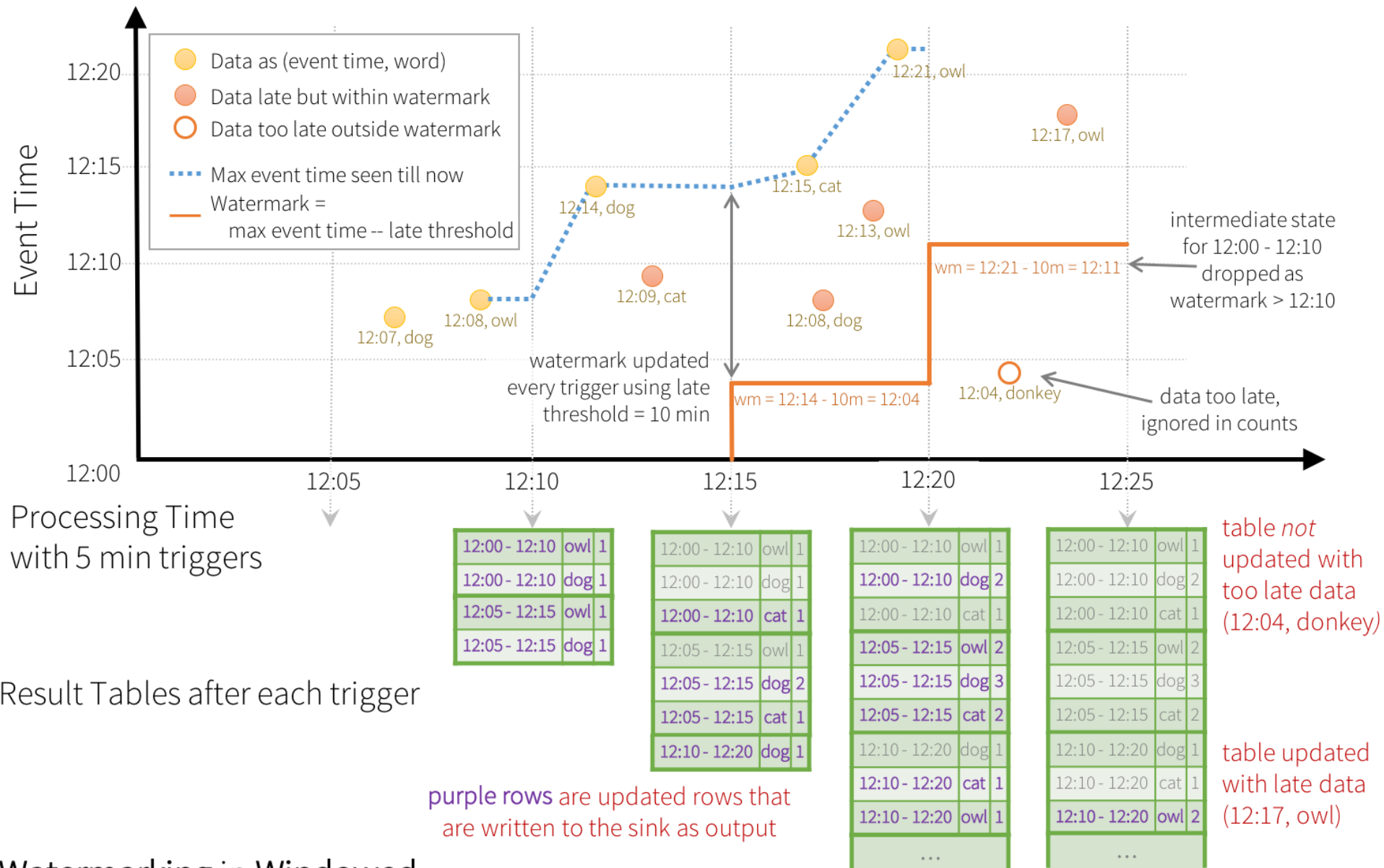
Code available on github!

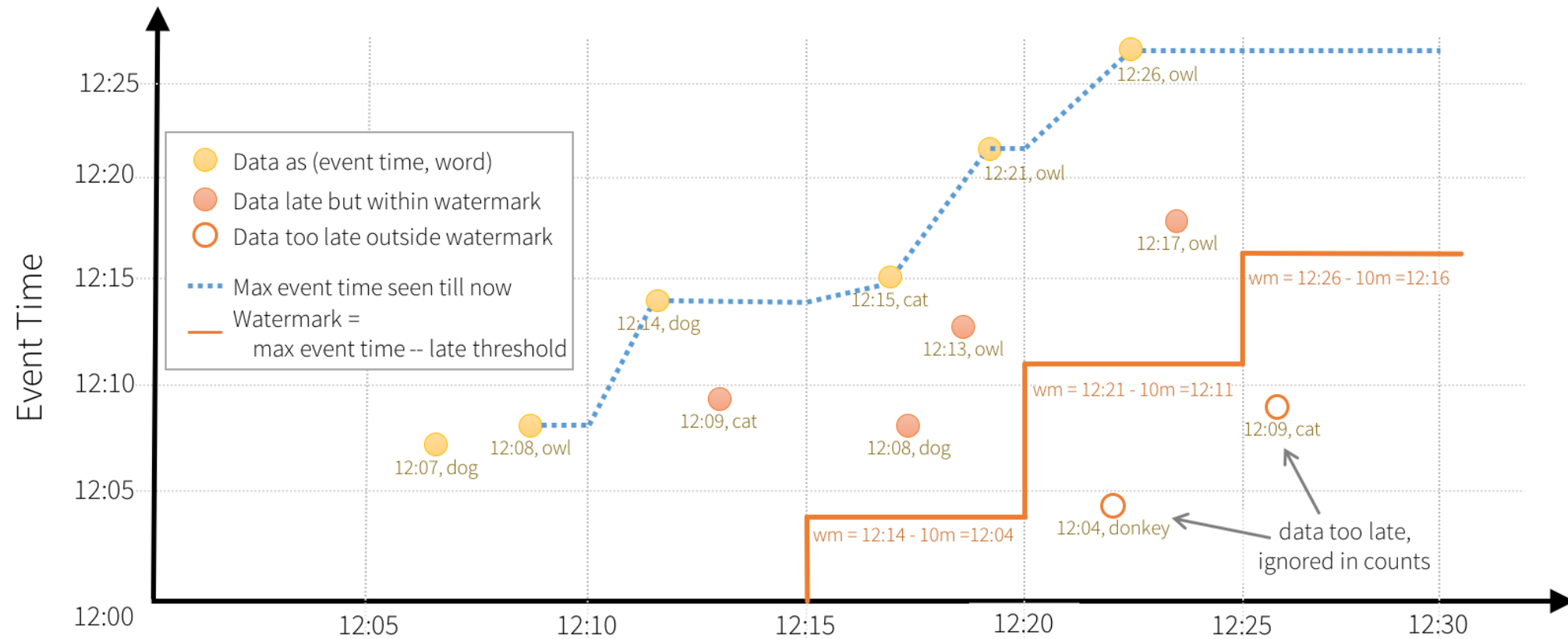
- All the code described in this lecture is available at

https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/Spark/sss_late-arrivals



- **NOTE:** It requires HW (8 cores and 16-32GB RAM), docker, docker-compose and a good internet connection. **Don't try this for the first time in class!**





partial counts for window 12:00 - 12:10 maintained as internal state while waiting for late data, so not yet added to result table

final counts for 12:00 - 12:10 added to table when watermark > 12:10, late data counted, and intermediate state for window dropped

12:00 - 12:10	owl	1
12:00 - 12:10	cat	1
12:00 - 12:10	dog	2

12:00 - 12:10	owl	1
12:00 - 12:10	cat	1
12:00 - 12:10	dog	2
12:05 - 12:15	owl	2
12:05 - 12:15	cat	2
12:05 - 12:15	dog	3

Watermarking in Windowed Grouped Aggregation with Append Mode

Result Tables after
each trigger

Semantic Guarantees of Aggregation with Watermarking

- A watermark delay of "X hours" guarantees that the **engine will never drop any data that is less than x hours late.**
- However, **data later than X hours is not guaranteed to be dropped**
 - it may or may not get aggregated
 - the more delayed, the less likely

Unsupported Operations

- Aggregation without grouping (e.g., counting rows of a table)
- OrderBy operations unless after aggregation and in Complete Output Mode
- Limit and take the first N rows
- Distinct operations
- Chaining multiple stateful operations in Update and Complete mode
- Few types of outer joins (see previous discussions)

Spark Structured Streaming

Emanuele Della Valle

Prof @ PoliMI

CRO & founder @ Motus ml

founder @ Quantia Consulting



POLITECNICO
MILANO 1863